# TD/TP6: Programmation dynamique – Correction

**CPES ≪ Sciences des données, arts et cultures ≫**

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

06 janvier 2026

## Exercice 1 : Empilement de tuiles

### Question 1

Deux critères majeurs justifient l'utilisation de la programmation dynamique :

— Sous-structures optimales : il est possible de déterminer la solution du problème à partir de solutions optimales du sous-problème. On comprend bien ici qu'on peut déterminer le nombre de tours différentes avec $(n, m, k)$ en comptant le nombre de tours obtenues dans des sous-configurations : $(n, m-1, k)$, $(n-1, m-1, k)$, etc. Il suffit ensuite de les sommer.

— Sous-problèmes répétés : certains sous-problèmes vont être rencontrés de multiples fois. En effet, pour déterminer le nombre de tours avec $(n, m, k)$, il va falloir déterminer le nombre de tours que l'on peut réaliser avec des largeurs de tuiles et des hauteurs de tours inférieures de façon récursive, qui mènera à la rencontre de sous-problèmes communs.

### Question 2

La stratégie repose sur les caractéristiques du problème : il n'est possible de poser une nouvelle tuile que sur une tuile autant ou plus large. Ainsi, pour s'assurer de ne pas rater de combinaisons, il faut partir de la tuile la plus large, et progressivement diminuer la largeur des tuiles.

Attention toutefois : il est possible d'utiliser $k$ fois chaque largeur de tuile. Or, comme ces tuiles doivent être posées par ordre décroissant de largeur, alors des tuiles de même largeur sont nécessairement empilées les unes sur les autres.

Ainsi, pour déterminer le nombre de tuiles avec une hauteur de $n$, une largeur maximale de $h$ et un nombre maximal de tuiles de même largeur de $k$, il suffit de compter le nombre de tours qui peuvent être construites :

— En ne choisissant aucune tuile de largeur $m$. Cela consiste à résoudre le même problème avec les paramètres $(n, m-1, k)$.

— En choisissant une seule tuile de largeur $m$. Cela consiste à résoudre le même problème avec les paramètres $(n-1, m-1, k)$.

— En choisissant plusieurs tuiles de largeur $m$, sans dépasser $k$ tuiles de la même largeur, et sans dépasser $n$ non plus. En notant $l$ ce nombre de tuiles de largeur $m$ choisies, cela revient à résoudre le même problème avec les paramètres $(n-l, m-1, k)$

La formule de récursivité est alors :

$$N(n,m,k) = \sum_{l=0}^{\min(k,n)} N(n-l, m-1, k)$$

, $\min(n,k)$ permettant de traiter le cas éventuel où $k > n$.

Deux cas d'arrêt assurent la bonne terminaison, et surtout l'assurance d'obtenir le nombre de tours attendu (correction). Il faut d'abord identifier la situation dans laquelle on a obtenu une tour complète. Cette situation a lieu lorsque l'on a déjà ajouté $n$ tuiles à la tour, et donc au moment de l'appel $N(0,m,k)$ pour n'importe quel $m$ et $k$. Ainsi, $N(0,m,k) = 1$. Il faut enfin arrêter d'essayer de construire une tour non-vide lorsque plus aucune tuile n'est disponible. Cela se traduit concrètement pas $N(n,0,k) = 0$ pour tout $n$ et $k$.

Ceci nous donne finalement :

$$N(n,m,k) = \begin{cases} 1 & \text{si } n = 0 \\ 0 & \text{si } m = 0 \\ \sum_{l=0}^{\min(k,n)} N(n-l, m-1, k) & \text{sinon} \end{cases}$$

## Question 3

Les algorithmes 1 et 2 présentent le pseudo-code de l'algorithme attendu.

## Question 4

L'idée est de toujours exploiter la relation de *récurrence*, mais de ne pas utiliser de *récursivité* dans l'algorithme (c'est à dire, pas d'appels à la fonction elle-même).

Alors que l'Algorithme de la question 3 est « *top-down* » (on part de ce qu'on veut calculer, et on fait des appels aux sous-problèmes qu'il faut résoudre pour avoir le résultat souhaité), celui de cette question (Algorithme 3) est « *bottom-up* » : on part des cas de base, et on construit un tableau itérativement jusqu'à atteindre la solution souhaitée.

**Algorithme 1** Algorithme pour le problème de l'empilement de tuiles avec récursion

**Entrée:** $n$ la hauteur des tours, $m$ la largeur maximale des tuiles et $k$ le nombre maximal de tuiles par valeur de largeur.

**Sortie:** $N(n, m, k)$ le nombre de tours différentes pour les paramètres passés en entrée.

```
memo ← Tableau de dimension (n + 1) × (m + 1)

pour i ∈ {0, ..., n} faire
    pour j ∈ {0, ..., m} faire
        memo[i, j] ← −1
    fin pour
fin pour

Retourner tours_recursif(n, m, k, memo)
```

**Algorithme 2** tours_recursif)

**Entrée:** $n$ la hauteur des tours, $m$ la largeur maximale des tuiles et $k$ le nombre maximal de tuiles par valeur de largeur. `memo`, la structure de mémoïsation.

**Sortie:** $N(n, m, k)$ le nombre de tours distinctes pour les paramètres passés en entrée.

```
si memo[n, m] ≠ −1 alors
    Retourner memo[n, m]
fin si

si n = 0 alors
    Retourner 1
sinon si m = 0 alors
    Retourner 0
sinon
    nb_tours ← 0
    pour l ∈ {0, ..., min(n, k)} faire
        nb_tours ← nb_tours + tours_memoisation_recursif(n − l, m − 1, k, memo)
    fin pour
    memo[n, m] ← nb_tours
    Retourner nb_tours
fin si
```

**Algorithme 3** Algorithme pour le problème de l'empilement de tuiles sans récursion

**Entrée:** $n$ la hauteur des tours, $m$ la largeur maximale des tuiles et $k$ le nombre maximal de tuiles par valeur de largeur.

**Sortie:** $N(n, m, k)$ le nombre de tours différentes pour les paramètres passés en entrée.

```
memo ← Tableau de dimension (n + 1) × (m + 1)

pour i ∈ {0, ..., n} faire
    pour j ∈ {0, ..., m} faire
        memo[i, j] ← 0
    fin pour
fin pour

pour i ∈ {0, ..., m} faire
    memo[0, i] ← 1
fin pour

pour i ∈ {1, ..., n} faire
    pour j ∈ {1, ..., m} faire
        nb_tours ← 0
        pour l ∈ {0, ..., min(n, k)} faire
            nb_tours ← nb_tours + memo[n − l, m − 1]
        fin pour
        memo[i, j] ← nb_tours
    fin pour
fin pour

Retourner memo[n, m]
```

### Question 4

L'analyse de l'algorithme 3 se fait rapidement :

— La ligne 1 se fait dans le pire des cas en $O(n \times m)$.

— Les lignes 2 à 6 présentent deux boucles pour imbriquées, la première itérant $n$ fois et la seconde $m$ fois. Ce qui est exécuté pour chacune des $n \times m$ itérations est en $O(1)$ : le tout est en $O(n \times m)$.

— Les lignes 7 à 9 présentent une unique boucle pour à $m$ itérations, ici aussi avec des opérations en $O(1)$ : le tout est en $O(m)$.

— Les lignes 10 à 18 présentent trois boucles pour imbriquées. La première fait $n$ itérations, la deuxième $m$ et la troisième $\min(n, k)$. Les opérations au sein des boucles sont toutes en $O(1)$ : le tout est en $O(\min(n, k) \times n \times m)$.

Le total est donc en $O(n \times m) + O(n \times m) + O(m) + O(\min(n, k) \times n \times m) = O(\min(n, k) \times n \times m)$. Ainsi on obtient $O(k \times n \times m)$ si $k < n$ (la situation la plus probable), et $O(n^2 \times m)$ sinon. La complexité temporelle est en $O(n \times m)$, à savoir la taille du tableau créé pour enregistrer les valeurs qui ont déjà été calculées. Elle est par ailleurs la même que la version récursive : dans les deux cas il faut allouer l'espace pour la structure de mémoïsation.

## Exercice 2 : Problème du sac à dos

## Question 6

Commençons par les cas de base. La complétion du sac à dos se termine dans deux cas : soit la capacité du sac est nulle, et on ne peut ajouter de nouveaux objets (qu'il en reste ou non), soit il reste de la capacité disponible, mais tous les objets ont été considérés et testés.

En notant $V(C, k)$ la valeur totale du sac pour une capacité $C$ et pour tous les objets d'indice 0 à $k$, on a pour le premier cas $V(0, k) = 0$ et $V(C, 0) = 0$ pour le second.

Passons maintenant au cas général. Comme pour l'empilement de tuile, on se place à une capacité (restante) donnée et un indice d'objet donné. Deux actions sont alors possibles : soit on ajoute l'objet au sac, auquel cas la valeur sera $v_k + V(C - p_k, k - 1)$, soit on décide de ne pas l'ajouter, et la valeur sera $V(C, k - 1)$. On veut choisir l'option qui maximise la valeur du sac, le cas général s'écrit donc : $V(C, k) = \max(v_k + V(C - p_k, k - 1), V(C, k - 1))$.

Attention toutefois : ce cas général n'a de sens que si $p_k \leqslant C$ ! Il faut donc ajouter un cas spécial pour les objets qui si ajoutés iraient au delà de la capacité restante. En combinant l'ensemble des cas (de base et généraux), on obtient :

$$V(C,k) = \begin{cases} 0 & \text{si } C = 0 ou k = 0 \\ V(C, k-1) & \text{si } p_k > C \\ \max(v_k + V(C - p_k, k-1), V(C, k-1)) & \text{sinon} \end{cases}$$

### Question 7

Voir le script Python joint sur Moodle (version non récursive).

### Question 8

On a deux boucles impbriquées : l'une itérant sur $C$, et l'autre sur $k$ : on a donc une complexité temporelle au pire cas en $P\left(C \times k\right)$.